

Deep Neural Network-based Dual Watermarking for Image Security

CHENTAO FAN*, cxf373, Case Western Reserve University, USA

YIHAO ZHANG*, yxz3114, Case Western Reserve University, USA

A final report for CSDS490 final project.

Additional Key Words and Phrases: Python, Machine Learning

1 Project Overview

1.1 Overview

The project aims to reconstruct a deep learning-based dual watermarking technique for image copyright protection and authentication. The technique will embed two types of watermarks into digital images: a perceptual hash for copyright protection and a cryptographic hash for content and source authentication. And we will also implement an Android application to show how to use our model into a real-world application.

1.2 Background

The rapid advancement of digital technology in the domain of image processing has paved the way for creating high-resolution digital content. Side-by-side, this advanced technology also helps the attacker to launch attacks on digital content, such as illegally manipulating, duplicating, or accessing information without the owners' knowledge, violating the integrity, authenticity, and copyright protection of the target digital content. Therefore, developing secure copyright protection and authentication has become a challenging task.

Digital watermarking is an effective technique for achieving copyright protection and authentication. The process of digital watermarking involves two primary operations: watermark embedding and watermark extraction.

In general, there are two types of digital watermarks addressed in the existing literature: visible and invisible watermarks. A visible watermark typically contains a visible message or a logo as a watermark in digital content. On the other hand, in invisible watermarking, a watermark is embedded and remains invisible, such that the digital media appears visually similar to the original image.

Invisible watermarks can be broadly classified into fragile and robust watermarks. Fragile watermarking is used in applications where even a single-bit change in the content is not allowed, as the single-bit change in the digital content will alter the watermark and will make it invalid. Therefore, fragile watermarking is used in digital content authentication and integrity verification. Robust watermarking shows robustness against content-preserving image manipulations, such as resizing, rotation, flipping, format conversion, etc, which makes it the most suitable watermarking technique for copyright protection. However, the degree of robustness is application-specific.

The most prominent technique for calculating the perceptual hash is the Discrete Cosine Transformation (DCT) technique. In this technique, the input image is expressed in terms of cosine functions oscillating at different frequencies. Frequency coefficients representing the lower frequency components contain important features of an image that are robust against distortions.

*Both authors contributed equally to this project.

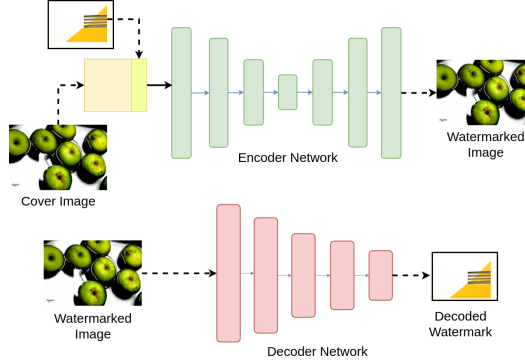


Fig. 1. General overview of deep learning-based watermarking where an encoder network is used for embedding the watermark and the decoder network is used for extracting the watermark.

2 Problem Statement

2.1 Problem

Deep learning-based watermarking techniques are the newest approach that can efficiently watermark images by overcoming the drawbacks of traditional watermarking techniques [3, 6, 13, 32]. In deep learning-based watermarking, encoder and decoder networks are used to perform watermarking, as shown in Figure 1. The encoder network constructs a watermarked image by embedding the watermark into the cover image, as shown in the upper part of Figure 1. Once the watermarked image is constructed, the decoder network (shown at the bottom part of Figure 1) is used to extract the watermark from the watermarked image.

Deep learning-based watermarking has its own challenges. The fragile nature of *DNN* can sometimes cause the model to fail [18]. As a result, fully utilizing the ability of deep neural networks to learn automatically and generalizing it to both watermark embedding and extracting processes is difficult. The lack of a good balance of imperceptibility and robustness is also a major concern in deep learning-based watermarking [1, 15, 19, 25, 32].

2.2 Example

Traditional image watermarking methods like DCT are not robust against content-preserving image manipulations and lack inherent watermark verification. Deep learning-based watermarking techniques, while more robust, still face challenges in maintaining a balance between robustness and imperceptibility and are vulnerable to overwriting and surrogate model attacks.

Relevant examples to emphasize the problem:

- Rotation or blurring of an image can distort embedded watermarks in traditional methods.
- Overwriting attacks can replace the original watermark with an attacker's watermark, compromising copyright protection.

2.3 Solution

Existing solutions include traditional watermarking techniques and single watermarking deep learning methods. However, these solutions are limited in their robustness and adaptability, and they do not address the need for source authentication and protection against overwriting attacks.

We propose a deep learning-based invisible dual watermarking technique that has all the features mentioned above. This dual watermarking technique serves two objectives at once:

- Protecting digital content copyright
- Content and Source authentication

Perceptual hash is used as the first watermark for protecting digital content copyright, whereas cryptographic hash is used as the second watermark for content and source authentication.

3 Methodology

3.1 Hashing

Hashing is a well-known concept in the domain of cryptography [14], through which a unique digest of a given message is generated. Cryptographic hash functions are generally used in digital signatures, message authentication codes, and to check the integrity of a message, image, or digital file [8].

3.1.1 Cryptographic Hash. Secure Hash Algorithm (*SHA*) is one of the standard cryptographic hash functions with a strong avalanche effect, which means any tiny change to the input will lead to a significant change at the output (hash value) [4, 8, 14]. When it comes to image processing, cryptographic hash becomes less useful, specifically where certain perceptual content-preserving image manipulations, such as JPEG compression, noise, blurring, cropping, deformations, etc, are allowed as these manipulations will alter the hash value.

3.1.2 Perceptual Hash. In this line, the perceptual hash is a class of comparable hash functions that generate a distinct and unique hash using robust or invariant features present in the input image [5, 9, 20–22, 30] such that two visually similar images should produce the same perceptual hash value. There are four basic techniques for generating perceptual hash: frequency transformations, dimensionality reduction, statistical methods, and extracting the local features of an image [12].

The most prominent technique for calculating the perceptual hash is the Discrete Cosine Transformation (*DCT*) technique [7, 28, 29].

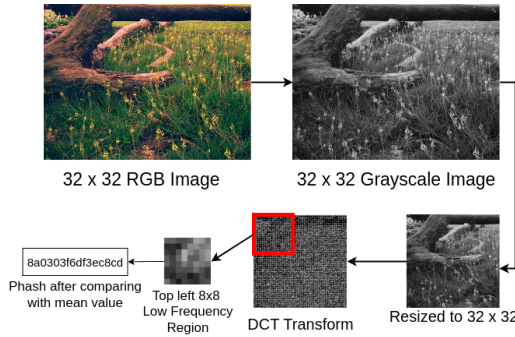


Fig. 2. Perceptual hash generation process

As shown in Figure 2, in this technique, the input image is expressed in terms of cosine functions oscillating at different frequencies. Frequency coefficients representing the lower frequency components contain important features of an image that are robust against distortions. In *DCT*-based perceptual hash, the input color image is first converted into a grayscale image and resized to 32×32 pixels. The grayscale image is then converted to the 32×32 frequency domain image through *DCT* transformation. The 8×8 block of pixels in the upper-left portion of the image represents the low-frequency block of the resized grayscale image. Each frequency coefficient present in the low-frequency block of pixels is compared to the mean value of all the frequency coefficients of

pixels in the low-frequency block. If the value of an individual frequency coefficient is greater than the mean value, its phase is taken as 1, and if it is not, it is taken as 0. The final hash is obtained by arranging the 64-bit binary values sequentially, thus representing the perceptual hash of an image.

Modification	SHA3 Hash	Perceptual Hash	Similarity	Image
Original Image	3970ad8d98939ec3...	91d3eae4662d904	100%	
Dot Modified Image	cbfb16ece37b3371...	91d3eae4662d904	100%	
Blurred Image	f569535b93f1243b...	91d3eae4662d904	100%	
Cropped Image	af448d68d5be9839...	99b5768aa1b172ca	76%	
Text Modified Image	4120d048987860d2...	91d3eae4662d904	100%	

Table 1. Example of perceptual hash values and SHA3 hash values for different images.



Fig. 3. Photoshop screenshot for dot modification

3.1.3 Example. The obtained perceptual hash will not vary if the image remains perceptually the same while SHA3 hash would not. As Table 1 shows, the perceptual hash value of the original image and the modified image is the same, while the SHA3 hash value is different.

For the dot modified image, the dot is modified using Photoshop as shown in Figure 3.

3.2 Watermark Embedding

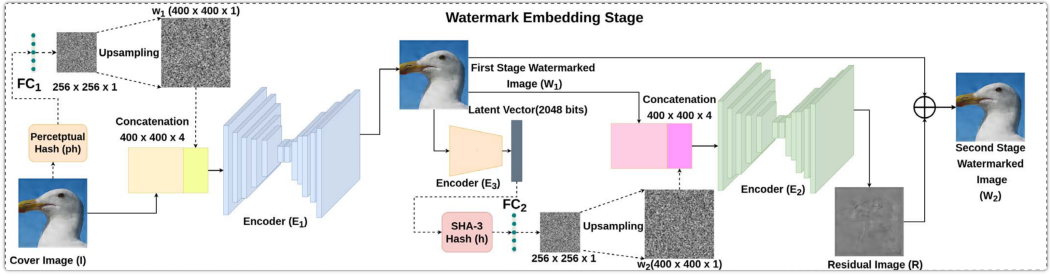


Fig. 4. Watermark embedding process

The watermark embedding process is shown in the Figure 4. The first step in our technique is to find the perceptual hash of the cover image I , which is our first watermark. There are different techniques for calculating the perceptual hash of an image. We have chosen DCT -based perceptual hash as discussed in subsubsection 3.1.2. It has advantages in terms of robustness against distortions and attacks caused by content-preserving image manipulation. We compute 64-bit DCT -based perceptual hash ph of I , which is passed into a pre-trained Fully Connected layer (FC) to produce $256 \times 256 \times 1$ tensor and then upsampled to a $400 \times 400 \times 1$ tensor. The upsampled tensor is concatenated to I as the fourth channel and passed to the encoder E_1 . The job of E_1 is to embed ph into I as a watermark and produce a $400 \times 400 \times 3$ image as output, which is our first stage watermarked image W_1 . It is ensured that W_1 will have the same perceptual hash as I by using three loss functions, which are residual regularization, LPIPS perceptual loss, and critic loss. More details regarding these loss functions are discussed in Section-3.4.

Next, we compute the 256-bit cryptographic hash h of W_1 using $SHA-3$ and use it as our second watermark. The h is passed through the same pre-trained FC to produce a $256 \times 256 \times 1$ tensor that is upsampled to $400 \times 400 \times 1$ tensor. The upsampled tensor is concatenated to W_1 as the fourth channel and $400 \times 400 \times 4$ tensor is given as input to the second encoder E_2 , which produces a residual image R of resolution $400 \times 400 \times 3$ as the output. The obtained R is added to W_1 , which yields the second stage watermarked image W_2 that is perceptually similar to the I . This addition of residual image (R) does not change the perceptual hash and perceptual similarity of W_2 with respect to I . This is ensured by using appropriate loss functions while training. Adding to that, we have also used two different techniques to embed the watermarks, thus reducing the chance of overlapping i.e., one watermark distorting the other. The technique followed for embedding h into W_1 is inspired by the approach of Stegastamp [26]. Dual watermarking makes it convenient to perform different verification in one watermarked image. In our case, W_2 protects digital content copyright, content and source authentication.

3.3 Watermark Extraction

The watermark extraction and verification process is shown in Figure 5, which consists of two independent extraction phases:

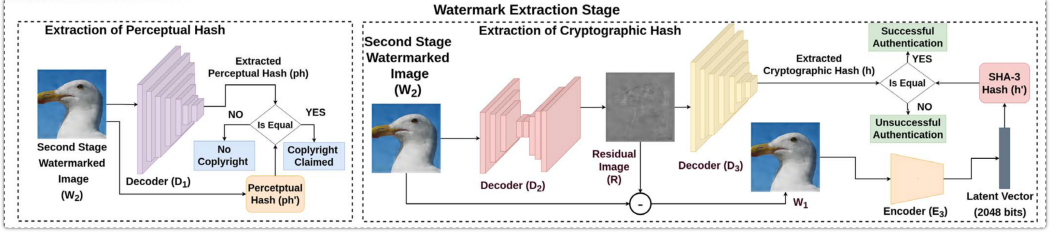


Fig. 5. Watermark extraction and verification process

- (1) *Extraction phase 1* is used for digital content ownership verification. We compute the perceptual hash (h') of W_2 using DCT transformation. Then D_1 extracts the embedded perceptual hash (h) from W_2 . If the extracted and the computed perceptual hashes of W_2 match, i.e., $ph = ph'$, the image ownership is claimed.
- (2) *Extraction phase 2* is used for content and source authentication for the given dual watermarked image W_2 . It uses two decoders: D_2 and D_3 . D_2 reconstructs a residual image (R') from given W_2 . R' is passed into D_3 , which decodes the cryptographic hash (h) present in R' . Subsequently, R' is subtracted from W_2 , and the SHA-3 hash value (h') of the result is computed. If the cryptographic hash (h) extracted by D_3 matches with the computed cryptographic hash (h'), i.e., $h = h'$, the authenticity of the content and source of the image is verified.

A robust copyright protection mechanism should be able to extract the watermark successfully from the watermarked image and claim ownership even if the watermarked image is tampered with. Hence, D_1 used in the *Extraction phase 1* should successfully extract the watermark from the tampered image to protect digital content copyright. To achieve this, we have used a DCT-based perceptual hash, which is robust against distortions, compression, white Gaussian noises, and Gaussian blurring. To verify the authenticity of the content and source for an image, the hash of the image should not change. Therefore D_2 and D_3 are used in the *Extraction phase 2*. It is ensured that D_2 fails to reconstruct the residual image if the image is tampered with. Thus both D_2 and D_3 are trained on the images without adding any distortions.

3.4 Network Architecture

Our model consists of five networks (E_1 , E_2 , D_1 , D_2 and D_3), which are trained in an end-to-end manner. We employ the U-Net [24] architecture for both E_1 and E_2 with skip connections between the encoder and decoder layers of each U-net. It helps E_1 and E_2 to generate watermarked images that are perceptually similar to the input cover image. Furthermore, the skip connections also preserve crucial spatial data and aid in recovering minute features that would have been lost during the downsampling of the image by the U-net encoders.

While embedding the perceptual hash, the emphasis is on the perceptual similarity between the reconstructed and original cover image, such that the perceptual hash does not change. In the same line, the hash is embedded and the residual image R is generated by E_2 , which is added to W_1 to produce W_2 .

We employ a spatial transformer [11] to extract the perceptual hash, which takes W_2 as an input and extracts its embedded perceptual hash ph . The spatial transformer used in D_1 provides invariance against various image transformations such as translation, scaling, rotation, cropping, deformations, change in brightness and contrast, etc. Decoding the cryptographic hash involves reconstructing R' using decoder D_2 and extracting h from R' using D_3 . U-Net is used to reconstruct

R' using D_2 , whereas we use a simple convolution neural network with five convolution layers followed by two fully connected layers for extracting the hash from R' using D_3 .

3.5 Loss Function and Scheme Objective

For each loss function below, in order to decrease the difference caused by different order of two comparable images, we use the mean value of each loss function with two directions. For example, for L_{PIPS} , we compute the perceptual similarity between I and W_1 as well as between W_1 and I , and then take the mean value of the two. The same applies to all other loss functions.

As mentioned earlier, the model is trained in an end-to-end manner. Generally, embedding watermarks distorts the image, however, in our case, the perceptual hash and cryptographic hash introduce a small distortion in the cover image (I) such that the perceptual hash remains the same. Embedding the first watermark requires I and the perceptual hash (ph) of I , which outputs W_1 . To ensure minimal distortion while embedding ph in I as a watermark, we used L_2 residual regularization (L_{MSE}), the LPIPS perceptual loss (L_{PIPS}) and a critic loss (L_C) functions. L_{MSE} reduces the chance of the encoder to overfit while L_{PIPS} and L_C increases the perceptual similarity between I and W_1 . L_{PIPS} help us to compute the perceptual similarity between the two images corresponding to a predefined network. We use VGG-19 in our case [31] as the predefined network. For given two images A and B , the L_{PIPS} corresponding to the VGG-19 is computed as follows:

$$L_{PIPS} = \sum_k \tau^k(VGG^k(A) - VGG^k(B)) \quad (1)$$

In Equation 1, features are extracted from k layers of the VGG-19 network. The function τ transforms deep embedding to a scalar LPIPS score. The final score between two images (A and B) is computed and averaged for k layers. L_C is calculated using a simple five-layer deep convolution neural network which classifies cover images and watermarked images using Wasserstein loss [2]. It predicts whether a watermark is encoded in an image such that the embedding of the watermark can be improved.

$$L_{E_1} = \lambda_1 L_{MSE}(I, W_1) + \lambda_2 L_{PIPS}(W_1, I) + \lambda_3 L_C(I, W_1) \quad (2)$$

Equation 2 refers to the loss function used for E_1 . Similarly, embedding the second watermark requires cryptographic hash (h) of W_1 and W_1 , which outputs W_2 . Thus we have used the same L_{MSE} , L_{PIPS} , and L_C loss for the encoder E_2 as referred in Equation 3.

$$L_{E_2} = \lambda_1 L_{MSE}(I, W_2) + \lambda_2 L_{PIPS}(W_2, I) + \lambda_3 L_C(I, W_2) \quad (3)$$

D_1 uses binary cross-entropy loss to extract the embedded perceptual hash (ph) from W_2 (Equation 4). Decoder D_2 is used for reconstructing R' from W_2 while decoder D_3 is used for extracting the embedded cryptographic hash (h) from R' . After reconstructing R' using D_2 it is compared with the original R using L_1 loss, L_{PIPS} loss, and Mean Square Error (L_{MSE}) (Equation 5). Then D_3 is used to extract h from R' , thus binary cross-entropy loss is used (Equation 6).

$$L_{D_1} = \lambda_7 L_{BCE}(D_1(W_2), ph) \quad (4)$$

$$L_{D_2} = \lambda_4 L_1(R, R') + \lambda_5 L_{PIPS}(R, R') + \lambda_6 L_{MSE}(R, R') \quad (5)$$

$$L_{D_3} = \lambda_d L_{BCE}(D_3(R'), h) \quad (6)$$

Training loss(Equation 7) is the sum of all five loss components (L_{E_1} , L_{E_2} , L_{D_1} , L_{D_2} , and L_{D_3}), which is minimized and λ s are weight factors.

$$L = L_{E_1} + L_{E_2} + L_{D_1} + L_{D_2} + L_{D_3} \quad (7)$$

3.6 Training of network

Our technique consists of two encoders and three decoders, which are trained together, as explained in subsection 3.4. To ensure that the proposed technique works, we need to minimize the loss functions such that the model converges smoothly. The training procedure starts with embedding both the perceptual and cryptographic hash as watermarks into the cover image, as explained in subsection 3.2. This is followed by the extracting phase, as discussed in subsection 3.3 for protecting digital content copyright as well as content and source authentication.

In order to protect digital content copyright, D_1 should extract the watermark even if W_2 is subject to content-preserving image manipulation such as JPEG compression, noise, blurring, cropping, deformations, etc. Hence, while training, we performed random blurring, adding random noise, flipping, rotating, cropping, and compression to random watermarked images in the datasets such that D_1 can be robust in extracting the embedded perceptual hash. To reduce the impact of resizing and cropping on watermarked images, we have restricted the embedding of perceptual hash in the centre portion of the images while training [26]. Decoders D_2 and D_3 are trained on the watermarked images without adding any distortions. This is due to the fact that minute changes in the watermarked image should be detected for successful content and source authentication. We have trained three instances (*CelebA*, *Mirlickr* and combined 200k images) of our model for 200 epochs with a batch size of 32. Early stopping and 10-fold cross-validation are employed to prevent the model from overfitting. We have used Adam optimizer to optimize the parameters of all five sub-models in our technique. The learning rate is set as 1.0×10^{-5} , and we decay the learning rate by 0.2 if the loss does not decrease within 10 epochs.

3.7 Android Application

We will implement an Android application to demonstrate the use of our model in a real-world application. The application will allow users to upload images, apply the watermarking technique, and verify ownership and authenticity. The app will be developed using Kotlin and the Android SDK, and it will provide a user-friendly interface for interacting with the watermarking model.

We have only implemented a simple Android application that allows users to upload images and apply the watermarking technique. The app will also provide options for verifying ownership for the owner. Which is to say that only the owner can verify the ownership of the image to check if the image belongs to him/her. The app will be developed using Kotlin and the Android SDK, and it will provide a user-friendly interface for interacting with the watermarking model.

Thanks to PyTorch Mobile, we can run the model on Android devices. The model will be converted to a format compatible with Android using the PyTorch Mobile library, which allows us to run PyTorch models on mobile devices.

4 Tools and Technologies

4.1 Tools and Technologies

The project will be implemented using the following tools and technologies:

- **Programming Languages:** Python, Kotlin, Java
- **Libraries:** TensorFlow, OpenCV, NumPy, Matplotlib, PyTorch, Pillow, Android SDK
- **Frameworks:** TensorFlow, PyTorch

- **Tools:** NVIDIA GPUs CUDA SDK for training, Visual Studio Code for report writing, Git for version control, Android Studio for Android application development, Trello for task management, Discord for communication

4.2 Dataset

We currently only used the Mirflickr dataset [10] for training and testing the model. It consists of different types of images with varied contexts, lighting, and themes, perfect for exhibiting our techniques' generalization. The images are of resolution 500×500 , which is downsampled to 400×400 for training.

To explore the generality of our technique, we used to want our evaluations on two datasets, CelebA [16] and Mirflickr [10]. It is due to the time limitation that we only used the Mirflickr dataset for training and testing the model. The CelebA dataset is a large-scale face attribute dataset with more than 200,000 celebrity images, each with 40 attribute annotations. It contains a wide variety of facial images with different poses, expressions, and backgrounds, making it suitable for evaluating the robustness of our watermarking technique.

5 Timeline

For now, we have the following milestones planned for the project:

- **Milestone 1:** Finished early stage paper reading and collection (Completed: March 15, 2025)
- **Milestone 2:** Reached out to the author and got source code (Completed: March 25, 2025)
- **Milestone 3:** Implement the dual watermarking model and test(Incompleted)
- **Milestone 4:** Implement DeepRAFT and generate ideal pretrained model (Completed: April 15, 2025)
- **Milestone 5:** Implement the Android application (Completed: April 20, 2025)
- **Milestone 6:** Compare the performance of our model with existing techniques (Incompleted)

Compared to the original timeline, we have made multiple changes to the timeline.

Since on the early stage of the project, we have not fully thought about how the project will be implemented and what is exactly needed, we have thought everything linearly and we have not thought about the exact time for each step. It was until we read the code and tried to reconstruct the model that we realized that this is not what we thought to be and far more complicated. Since it is our first time fully implementing a neural network model processing images we have to spend a lot of time reading each function and compare them with different papers, codes and the original training process. But we have learned a lot from this process and we are very happy that we have done it. Even though we have not fully implemented the dual watermarking model, we have learned a lot on its structure and the way it works. Since this summer is a long break, we decided to take further research on the dual watermarking model and try to implement it in the future.

6 Outcomes

We intended to achieve the following objectives with our project:

- Develop a dual watermarking technique for image copyright protection and authentication.
- Implement an Android application to demonstrate the use of our model in a real-world application.
- Evaluate the performance of our model on different datasets and compare it with existing techniques.

However we have not yet completed the implementation of the dual watermarking technique, we have reached out to the original author of the paper and got their code. But the code is not the same as the paper, and some of the crucial parts are not implemented yet.

But to make sure we can finish the project on time, we have implemented another model called DeepRAFT (<https://github.com/xinghua-qu/DeepRAFT>) and forked the code from the original author. After some modifications, we have successfully implemented the model and got the results and further implement the model into an Android application.

6.1 Results

6.1.1 Dual Watermarking. Here is some running results of our implementation of dual watermarking:

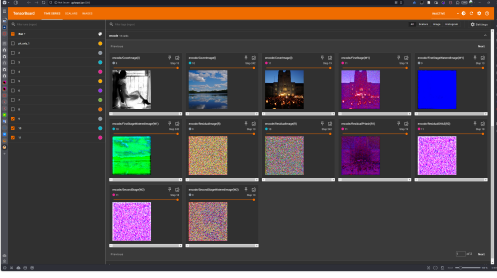


Fig. 6. Dual watermarking generated image

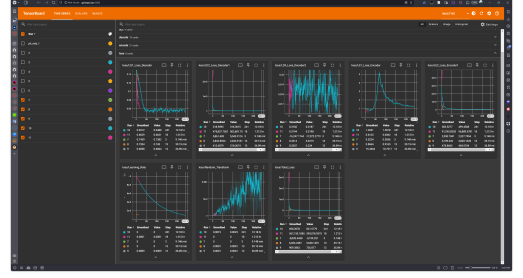


Fig. 7. Dual watermarking loss curve

6.1.2 DeepRAFT. Here is some running results of our implementation of DeepRAFT:

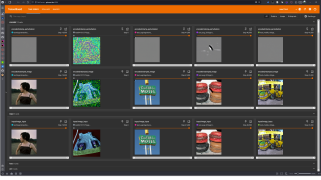


Fig. 8. DeepRAFT image



Fig. 9. DeepRAFT loss curve

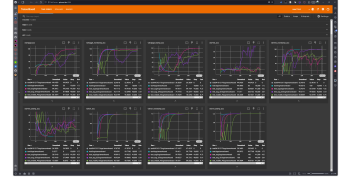


Fig. 10. DeepRAFT acc curve

6.1.3 Android Application. Here is some running screenshots of our implementation of Android application:

6.2 Code Repository

The code for the project is hosted on GitHub at the following link:

- *Dual Watermarking* <https://github.com/Aphcity/dual-channel-watermarking>
- *DeepRAFT Watermarking* <https://github.com/Aphcity/DeepRAFT>
- *Android Application* https://github.com/YihaoCWRU/Watermark_Android

7 Further Work

Since we have not yet completed the implementation of the dual watermarking technique, but we have learned a lot from other models like DeepRAFT and Stegastamp, we will further implement the dual watermarking technique and compare the performance of our model with existing techniques. To make the next coming steps more clear, we have listed the following steps:

- Implement Encoder 2 and Decoder 2 for the dual watermarking model

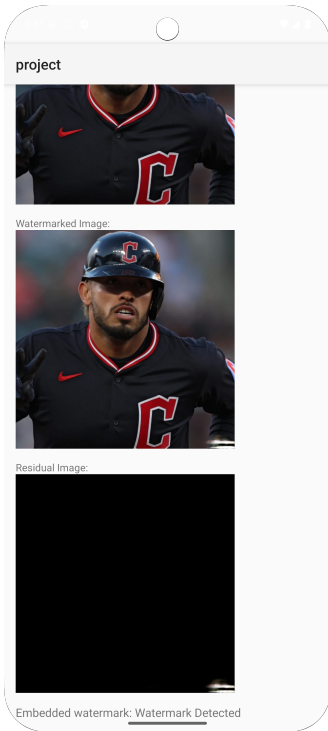


Fig. 11. Watermark Embedded

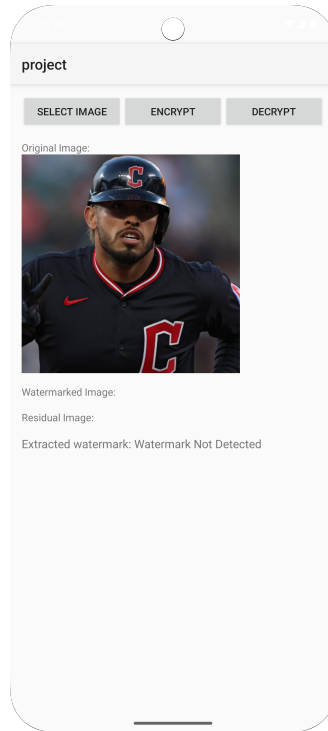


Fig. 12. No watermark detected

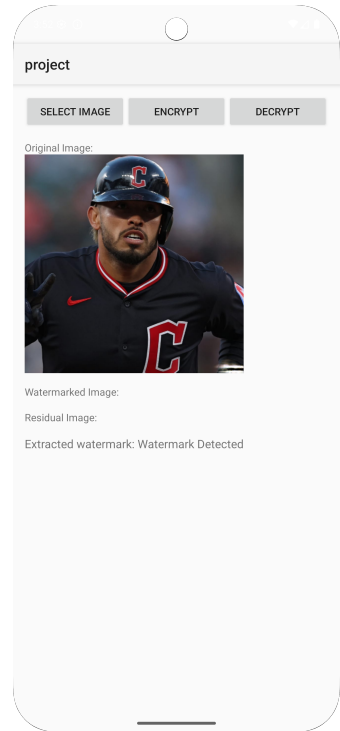


Fig. 13. Watermark detected

- Implement the loss function and training the model for the dual watermarking model where we think the loss function of Encoder 1 and Encoder 2 should be further modified to make sure the watermarked image is perceptually similar to the original image.
- Train the auto encoder model to get the hash latent vector and then use the hash latent vector to generate the residual image.
- Further examine the whole watermarking process and make sure it is reasonable and robust.
- Try to further implement the Android application and make it more user-friendly and can use camera to scan multiple images lively.
- Compare the performance of our model with existing techniques and make sure it is robust and can be used in real-world applications.

8 Work Division

For specific tasks, we have finished following tasks:

For Chentao Fan:

- Collected latest papers, codes and reading them
- Reached out to the original author and getting the source code
- Implemented Encoder 1 and Decoder 1
- Implemented Perceptual Hash and verify its correctness
- Implemented Loss function and training the model
- Implemented thorough logging system to record the training process using Tensorboard

- Compatible with PyTorch Mobile and convert the model to a format compatible with Android

For Yihao Zhang:

- Prepared the dataset
- Implemented Stegastamp and generated ideal pretrained model(ABANDONED, not able to optimize for mobile since it is TensorFlow model)
- Implemented DeepRAFT and generated ideal pretrained model
- Implemented VINE and generated ideal pretrained model(ABANDONED, not able to get the code and optimize for mobile)
- Implemented the Android application
- Prepared the presentation

Compared to the original plan, we have made multiple changes to the work division.

we have originally planned to implement the dual watermarking model, a DCT based, and a DWT based model. And do the comparison between the three models. But due to the time limitation, we have only implemented the dual watermarking model and it is still not fully implemented yet. So we have implemented another model called DeepRAFT [23] and forked the code from the original author. After some modifications, we have successfully implemented the model and got the results and further implement the model into an Android application. Meanwhile, we have also tested some other models like Stegastamp and VINE, but since Stegastamp [27] is a TensorFlow model and not able to optimize for mobile, we have abandoned it. As for VINE, even the model is published on Huggingface and according to the paper [17], we are not able to get the code and optimize for mobile, so we have abandoned it as well.

References

- [1] Mahdi Ahmadi, Alireza Norouzi, Nader Karimi, Shadrokh Samavi, and Ali Emami. 2020. ReDMark: Framework for residual diffusion watermarking based on deep networks. *Expert Systems with Applications* 146 (2020), 113157.
- [2] Martin Arjovsky, Soumith Chintala, and Léon Bottou. 2017. Wasserstein generative adversarial networks. In *International conference on machine learning*. PMLR, 214–223.
- [3] Shumeet Baluja. 2019. Hiding images within images. *IEEE transactions on pattern analysis and machine intelligence* 42, 7 (2019), 1685–1697.
- [4] Zhuozhao Chen and Guodong Ye. 2022. An asymmetric image encryption scheme based on hash SHA-3, RSA and compressive sensing. *Optik* 267 (2022), 169676.
- [5] Ling Du, Anthony TS Ho, and Runmin Cong. 2020. Perceptual hashing for image authentication: A survey. *Signal Processing: Image Communication* 81 (2020), 115713.
- [6] Han Fang, Dongdong Chen, Qidong Huang, Jie Zhang, Zehua Ma, Weiming Zhang, and Nenghai Yu. 2020. Deep template-based watermarking. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 4 (2020), 1436–1451.
- [7] Mengjuan Fei, Jing Li, and Honghai Liu. 2015. Visual tracking based on improved foreground detection and perceptual hashing. *Neurocomputing* 152 (2015), 413–428.
- [8] G.L. Friedman. 1993. The trustworthy digital camera: restoring credibility to the photographic image. *IEEE Transactions on Consumer Electronics* 39, 4 (1993), 905–910. doi:10.1109/30.267415
- [9] Ziqing Huang and Shiguang Liu. 2021. Perceptual Image Hashing With Texture and Invariant Vector Distance for Copy Detection. *IEEE Transactions on Multimedia* 23 (2021), 1516–1529. doi:10.1109/TMM.2020.2999188
- [10] Mark J Huiskes and Michael S Lew. 2008. The mir flickr retrieval evaluation. In *Proceedings of the 1st ACM international conference on Multimedia information retrieval*. 39–43.
- [11] Max Jaderberg, Karen Simonyan, Andrew Zisserman, et al. 2015. Spatial transformer networks. *Advances in neural information processing systems* 28 (2015).
- [12] Xinran Li, Chuan Qin, Zichi Wang, Zhenxing Qian, and Xinpeng Zhang. 2022. Unified performance evaluation method for perceptual image hashing. *IEEE Transactions on Information Forensics and Security* 17 (2022), 1404–1419.
- [13] Yue Li, Hongxia Wang, and Mauro Barni. 2021. A survey of deep neural network watermarking techniques. *Neurocomputing* 461 (2021), 171–193.
- [14] Ching-Yung Lin and Shih-Fu Chang. 1998. Generating robust digital signature for image/video authentication. In *Multimedia and Security Workshop at ACM Multimedia*, Vol. 98. Citeseer, 49–54.

- [15] Yang Liu, Mengxi Guo, Jian Zhang, Yuesheng Zhu, and Xiaodong Xie. 2019. A novel two-stage separable deep learning framework for practical blind watermarking. In *Proceedings of the 27th ACM International conference on multimedia*. 1509–1517.
- [16] Ziwei Liu, Ping Luo, Xiaogang Wang, and Xiaoou Tang. 2015. Deep Learning Face Attributes in the Wild. In *Proceedings of International Conference on Computer Vision (ICCV)*.
- [17] Shilin Lu, Zihan Zhou, Jiayou Lu, Yuanzhi Zhu, and Adams Wai-Kin Kong. 2025. Robust Watermarking Using Generative Priors Against Image Editing: From Benchmarking to Advances. doi:10.48550/arXiv.2410.18775 arXiv:2410.18775 [cs]
- [18] Xiyang Luo, Ruohan Zhan, Huiwen Chang, Feng Yang, and Peyman Milanfar. 2020. Distortion agnostic deep watermarking. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13548–13557.
- [19] Seung-Min Mun, Seung-Hun Nam, Haneol Jang, Dongkyu Kim, and Heung-Kyu Lee. 2019. Finding robust domain from attacks: A learning framework for blind watermarking. *Neurocomputing* 337 (2019), 191–202.
- [20] Xia-mu Niu and Yu-hua Jiao. 2008. An overview of perceptual hashing. *ACTA ELECTONICA SINICA* 36, 7 (2008), 1405.
- [21] Arun Qamra, Yan Meng, and Edward Y Chang. 2005. Enhanced perceptual distance functions and indexing for image replica recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 27, 3 (2005), 379–391.
- [22] Chuan Qin, Enli Liu, Guorui Feng, and Xinpeng Zhang. 2020. Perceptual image hashing for content authentication based on convolutional neural network with multiple constraints. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 11 (2020), 4523–4537.
- [23] Xinghua Qu, Alvin Chan, Yew-Soon Ong, Pengfei Wei, Xiang Yin, Caishun Chen, Zhu Sun, and Zejun Ma. 2022. Only For You: Deep Neural Anti-Forwarding Watermark Preserves Image Privacy. (Sept. 2022).
- [24] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. 2015. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2015: 18th International Conference, Munich, Germany, October 5–9, 2015, Proceedings, Part III* 18. Springer, 234–241.
- [25] Weiwei Sun, Jiantao Zhou, Yuanman Li, Ming Cheung, and James She. 2020. Robust high-capacity watermarking over online social network shared images. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 3 (2020), 1208–1221.
- [26] Matthew Tancik, Ben Mildenhall, and Ren Ng. 2020. Stegastamp: Invisible hyperlinks in physical photographs. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2117–2126.
- [27] Matthew Tancik, Ben Mildenhall, and Ren Ng. 2020. StegaStamp: Invisible Hyperlinks in Physical Photographs. doi:10.48550/arXiv.1904.05343 arXiv:1904.05343 [cs]
- [28] Zhenjun Tang, Fan Yang, Liyan Huang, and Xianquan Zhang. 2014. Robust image hashing with dominant DCT coefficients. *Optik* 125, 18 (2014), 5102–5107.
- [29] Huan Wang and Hongxia Wang. 2018. Perceptual hashing-based image copy-move forgery detection. *Security and Communication Networks* 2018 (2018), 1–11.
- [30] Christoph Zauner. 2010. Implementation and benchmarking of perceptual image hash functions. *na* (2010).
- [31] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. 2018. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 586–595.
- [32] Xin Zhong, Pei-Chi Huang, Spyridon Mastorakis, and Frank Y Shih. 2020. An automated and robust image watermarking scheme based on deep neural networks. *IEEE Transactions on Multimedia* 23 (2020), 1951–1961.